

הרצאה 6 – Javascript המשך

בל הזכויות שמורות לקרן יעקב 2025

תוכן עניינים

1. תחביר השפה (סינטקס) & סדר, משמעות ומוסכמות
2. מבני נתונים
3. אירועים - events
4. ה-DOM
5. ES6



1. תחביר השפה (סינטקס) & סדר, משמעות ומוסכמות

תחביר השפה – Syntax



- שפת JavaScript היא cAse senSitivE – רגישה לאותיות קטנות ולגדולות.
- טקסט (שאינו מספר) צריך להיות מוכל בתוך גרשיים, או בתוך גרש בודד. יש להקפיד לא לבלבל בין השניים (פתיחה וסגירה זהים). גם אסור להשתמש באותו הסוג זה בתוך זה.
- הערות של שורה אחת כותבים על ידי הוספת // בתחילת השורה
- הערות בנות יותר משורה אחת מכילים בתוך /* */
- לא חובה לשים נקודה-פסיק בכל סוף שורה, אבל בקורס נשים תמיד נקודה פסיק בכל סוף שורה
- ב-JavaScript אין pointers !!
- References פועלים באופן שונה ממה שאתם מכירים משפות תכנות אחרות.
- ב-JavaScript לא ניתן לקבל reference ממשתנה אחד למשתנה אחר.
- רק אובייקט ומערך (אובייקטים מורכבים) מוקצים על ידי הפניה by-reference. אובייקטים פרימיטיביים מוקצים by-value.
- המשמעות בעמוד הבא...

שליחת משתנים לפונקציות

- **המשמעות** – משתנה מקומי שייך לפונקציה ואינו משתנה בפונקציה אחרת, אלא אם הוא גלובלי או מסוג אובייקט / מערך:
 - משמע, אם שולחים משתנה פרימיטיבי לפונקציה, זה העתק של המשתנה, והמשתנה המקומי בפונקציה המקורית לא השתנה
 - לעומת זאת, משתנה מסוג Object או מערך – מועבר reference אליהם ולכן כל שינוי בפונקציה אחרת כן ישנה אותם בפונקציה המקורית!

ערכי משתנים

- number
- bool (true and false)
- null & undefined
- string
- symbol
- object

typeof() – returns type of the variable

```
> a = 8
< 8
> typeof(a)
< 'number'
> |
```

```
> let str = "hello"
< undefined
> typeof(str)
< 'string'
>
```

```
> const boool = true
< undefined
> typeof(boool)
< 'boolean'
>
```

```
> let arr = [ 1, 2, "string1" ]
< undefined
> typeof(arr)
< 'object'
>
```

משתנים

● מכיוון שאנחנו לא מצהירים על סוגי משתנים ב-JS, אנחנו יכולים לכתוב למשל:
מה יהיה הפלט בכל אחד מהמקרים?

רמז - זאת לא תהיה שגיאה

```
8   const sumNumStr = "Volvo" + 16;  
9   console.log(sumNumStr);  
10  
11  let sumNumsStr = 16 + 4 + "Volvo";  
12  console.log(sumNumsStr);  
13  
14  let sumStrNum = "Volvo" + 16 + 4;  
15  console.log(sumStrNum);
```

משתנים

● מכיוון שאנחנו לא מצהירים על סוגי משתנים ב-JS, אנחנו יכולים לכתוב למשל:
מה יהיה הפלט בכל אחד מהמקרים?

רמז - זאת לא תהיה שגיאה

```
8  const sumNumStr = "Volvo" + 16;  
9  console.log(sumNumStr); // Volvo16  
10  
11  let sumNumsStr = 16 + 4 + "Volvo";  
12  console.log(sumNumsStr); // 20Volvo  
13  
14  let sumStrNum = "Volvo" + 16 + 4;  
15  console.log(sumStrNum); // Volvo164
```

למה? Javascript מחשב משמאל לימין, כך שהסדר יכול להשפיע על התוצאה.
בשמחברים מספר ומחרוזת, Javascript יתייחס למספר כאל מחרוזת

הצהרה על משתנים

- מאז ES6, משתנים מוצהרים על ידי let או const בתחום ה-block (ראינו בהרצאה הקודמת)
- אפשר לדרוס מספר עם string ולהיפך (אבל עדיף שלא)
- לפני ES6, ההצהרה היתה באמצעות var באופן גלובלי בעמוד, או לוקאלי בתוך פונקציה, ללא התייחסות לבלוק. שוב, אנחנו לא נשתמש ב-var בקורס, כדי למנוע טעויות לפני שהן קורות.

מה יודפס בכל אחד מקטעי הקוד?

```
8 let number1 = 6;  
9 number1 = 1;  
10 console.log(number1);  
11  
12 const number2 = 9;  
13 number2 = 2;  
14 console.log(number2);
```

```
16 console.log(number3);  
17 let number3 = 3;  
18  
19 console.log(number4);  
20 const number4 = 4;
```

הצהרה על משתנים (1)

- מאז ES6, משתנים מוצהרים על ידי `let` או `const` בתחום ה-`block` (ראינו בהרצאה הקודמת)
- אפשר לדרוס מספר עם `string` ולהיפך (אבל עדיף שלא)
- לפני ES6, ההצהרה היתה באמצעות `var` באופן גלובלי בעמוד, או לוקאלי בתוך פונקציה, ללא התייחסות לבלוק. שוב, אנחנו לא נשתמש ב-`var` בקורס, כדי למנוע טעויות לפני שהן קורות.

מה יודפס בכל אחד מקטעי הקוד?

```
8 let number1 = 6;
9 number1 = 1;
10 console.log(number1);
11
12 const number2 = 9;
13 number2 = 2;
14 console.log(number2);
```

```
1
✖ ▶ Uncaught TypeError: Assignment to constant variable.
   at vars.html:13:21
>
```

```
16 console.log(number3);
17 let number3 = 3;
18
19 console.log(number4);
20 const number4 = 4;
```

```
✖ ▶ Uncaught ReferenceError: Cannot access 'number3' before initialization
   at vars.html:16:25
>
```

הצהרה על משתנים (2)



```
8   const colors = ["yellow", "orange", "blue"];
9   colors.push("green");
10  console.log(colors);
11
12  colors = ["yellow", "orange", "blue", "pink"];
13  console.log(colors);
```

מה יודפס בכל אחד מקטעי הקוד?



```
15  var name = "Winnie-the-Pooh";
16
17  (function() {
18    console.log(name);
19    var name = "Winnie-the-Pooh";
20  })(); |
```

הצהרה על משתנים (3)

```
8   const colors = ["yellow", "orange", "blue"];
9   colors.push("green");
10  console.log(colors);
11
12  colors = ["yellow", "orange", "blue", "pink"];
13  console.log(colors);
```

▶ (4) ['yellow', 'orange', 'blue', 'green']

✖ ▶ Uncaught TypeError: Assignment to constant variable.
at vars.html:12:20

>

```
15  var name = "Winnie-the-Pooh";
16
17  (function() {
18  console.log(name);
19  var name = "Winnie-the-Pooh";
20  })(); |
```

undefined

>

שמות משתנים, פונקציות ואובייקטים

- שמות של משתנים הם מחרוזת של תווים ב-camelCase
- מותר להשתמש באותיות לועזיות בלבד, בספרות ובשני תווים מיוחדים: \$ - ו
- התו הראשון חייב להיות אות
- יש מילים שמורות שלא יכולות להיות שמות של משתנים, כמו var, let
- הצהרה על משתנים לפני השימוש בהם

- שמות לפונקציות camelCase
- פתיחת פונקציות- סוגריים ראשונים באותה השורה
- הצהרה על פונקציות לפני הקריאה להן, כך נחסוך את ה-hoisting

```
function setTimers () {  
    //some code goes here  
}  
a_el.onclick = function() {  
    setTimers();  
};
```

- שמות לאובייקטים Math, Date - אות ראשונה גדולה

תנאים - if

```
8      const isWinter = false;
9      let temper;
10
11      if (isWinter) {
12          temper = "cold";
13      }
14      else {
15          temper = "hot";
16      }
```

● תנאי - כשאנחנו מציינים `if (elem)` אנחנו בודקים שהמשתנה:

- לא `undefined`
- לא `null`
- לא מחרוזת ריקה
- לא `0` (אפס)
- לא `NaN`
- לא `false`

● מצד שמאל - אותו קוד, החלק התחתון יעיל יותר

```
18      const isWinter = false;
19      const temperature = isWinter ? "cold" : "hot";
```

Switch



- אפשר לבדוק תנאים גם ע"י ביטוי switch
- בודקים אם הערך שבסוגריים: switch(X) שווה לערכים אחרים, המופיעים ב- case
- לאחר כל הוראת case נרשום את כל ההוראות לביצוע ואז break, כדי למנוע מהקוד להיכנס ל- case-ים העוקבים.

```
8   let whoAmI = 4;
9
10  switch (whoAmI) {
11      case 4:
12          console.log("I am 4");
13          break;
14      case 5:
15          console.log("I am 5");
16          break;
17      case 6:
18          console.log("I am " + whoAmI);
19          break;
20  }
```

לולאת for



```
for (let i=1; i<=10; i++) {...}
```

- הלולאה for יוצרת לולאת קוד

- הפרמטרים של הלולאה:

- הפרמטר הראשון מציין את הערך התחילי של משתנה הלולאה
- הפרמטר השני הוא התנאי לסיום הלולאה
- הערך השלישי הוא הערך שבו יש לקדם את משתנה הלולאה בכל מחזור.

- ייעול: הוספת continue בנקודה שרוצים שהקוד ימשיך לאיטרציה הבאה

```
1 function initialize()
2 {
3     const main = document.getElementsByTagName("main")[0];
4     let articles = "";
5
6     for (i = 0; i < 10; i++) {
7         articles += "<article></article>";
8     }
9
10    main.innerHTML = articles;
11 }
```


לולאת for of



- מאפשרת לרוץ על מבני נתונים iterable כמו מערכים
- מאפשרת לרוץ על **כל** האובייקטים במערך בלי לחשב את גודל המערך

מה יהיה הפלט?

```
8 let arr = [, 2, , 4]; // iterative data type
9 arr[9] = 100;
10
11 for(const element of arr) {
12     console.log(element);
13 }
```

לולאת for of

- מאפשרת לרוץ על מבני נתונים iterable כמו מערכים
- מאפשרת לרוץ על **כל** האובייקטים במערך בלי לחשב את גודל המערך

מה יהיה הפלט?

```
8 let arr = [, 2, , 4]; // iterative data type
9 arr[9] = 100;
10
11 for(const element of arr) {
12     console.log(element);
13 }
```

```
undefined for of.html:12
2 for of.html:12
undefined for of.html:12
4 for of.html:12
5 undefined for of.html:12
100 for of.html:12
>
```

לולאת for in



● מאפשרת לרוץ על מבני נתונים arbitrary כמו אובייקטים ו- json

● ב- in המשתנה שרץ הוא המפתח

```
17 let obj = {name:"diana", role:"queen"}; // arbitrary data type
18
19 for(const key in obj){
20     console.log("key is " + key);
21     console.log("val is " + obj[key]);
22     console.log(`${key}: ${obj[key]}`);
23 }
```

מה יהיה הפלט?

לולאת for in

● מאפשרת לרוץ על מבני נתונים arbitrary כמו אובייקטים ו- json

● ב- in המשתנה שרץ הוא המפתח

```
17 let obj = {name:"diana", role:"queen"}; // arbitrary data type
18
19 for(const key in obj){
20     console.log("key is " + key);
21     console.log("val is " + obj[key]);
22     console.log(`${key}: ${obj[key]} `);
23 }
```

מה יהיה הפלט?

```
key is name
val is diana
name: diana
key is role
val is queen
role: queen
```

>

לולאת for in



```
25 let arr = [, 2, , 4];
26 arr[9] = 100;
27
28 for(const element in arr) {
29     console.log(element);
30 }
```

מה יודפס לקונסול?



לולאת for in

```
25 let arr = [, 2, , 4];  
26 arr[9] = 100;  
27  
28 for(const element in arr) {  
29   console.log(element);  
30 }
```

```
1  
3  
9  
>
```

● מה יודפס לקונסול?

- For in – הלולאה עוברת רק על מה שקיים ולא על כל המערך לפי הסדר
- שימו לב שאתם רואים פה את האינדקסים של המפתחות, ולא את הערכים!

אופרטורים

● אופרטורים הם סימנים שבהם אנו משתמשים לבדיקה ולקביעת ערך.

● == בדיקת שיוויון (ללא בדיקת סוג המשתנה אלא ערכו)

● === (בדיקת ערך וגם בדיקת סוג)

● != בדיקת אי-שיוויון ללא סוג

● !== בדיקת אי שיוויון סוג וגם ערך

● >= גדול או שווה

● <= קטן או שווה

● > גדול מ

● < קטן מ

● % מודולוס, מציין שארית של פעולת חילוק (לדוגמא: $2 = 23 \% 7$)

```
8 let a = 5;
9 let b = "5";
10
11 console.log(a == b);
12 console.log(a === b);
```

```
true
false
> |
```

אופרטורים לוגיים

אופרטורים לוגיים

- && וגם (צירוף של תנאים: כל התנאים צריכים להתקיים)
- || או (לפחות אחד מהתנאים צריך להתקיים)

פעולות חשבון

- + חיבור
- - חיסור
- += הוספת ערך לעצמו
- -= החסרת ערך מעצמו
- וכן"ל כפל, חילוק וכו'
- ++ הוספת 1 לעצמו
- -- החסרת אחד מעצמו

סדר קריאת הדף ע"י הדפדפן

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>JS - Order</title>
5     <script>
6       alert(document.getElementById("link").innerHTML);
7     </script>
8   </head>
9   <body>
10    <a href="#" id="link">Click here</a>
11  </body>
12 </html>
```

● הרצת הקוד הבא גורמת לשגיאה, למה?

```
✖ ▶ Uncaught TypeError: Cannot read properties of null (reading 'innerHTML')
   at order.html:6:50
> |
```

סדר קריאת הדף ע"י הדפדפן - המשך

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>JS - Order</title>
5     <script>
6       alert(document.getElementById("link").innerHTML);
7     </script>
8   </head>
9   <body>
10    <a href="#" id="link">Click here</a>
11  </body>
12 </html>
```

```
✖ ▶ Uncaught TypeError: Cannot read properties of null (reading 'innerHTML')
   at order.html:6:50
```

```
> |
```

- הרצת הקוד הבא גורמת לשגיאה, למה?
- הקוד עדיין לא מזהה את קיומו של link ולא מקפיץ את ה-alert או שמקפיץ אותו ריק (תלוי בדפדפן).
- הסיבה: הדפדפן קורא מלמעלה למטה: קודם את הקוד ואחר כך את קיומו של האובייקט link ב-body. לכן, כשנקראת פונקציית ה-alert, הדפדפן עדיין אינו יודע שבהמשך הדף יוגדר link.

סדר קריאת הדף ע"י הדפדפן - המשך

- נרשום את הקוד אחרת -
הקוד תקין!

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>JS - Order2</title>
5   </head>
6   <body>
7     <a href="#" id="link">Click here</a>
8
9     <script>
10      alert(document.getElementById("link").innerHTML);
11    </script>
12  </body>
13 </html>
```

127.0.0.1:5500 says

Click here

OK

סדר קריאת הדף ע"י הדפדפן - המשך

- ויש עוד אופציה - הקוד תקין!

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>JS - Order3</title>
5     <script>
6       window.onload = () => alert(document.getElementById("link").innerHTML);
7     </script>
8   </head>
9   <body>
10    <a href="#" id="link">Click here</a>
11  </body>
12 </html>
```

127.0.0.1:5500 says

Click here

OK

האירוע load נורה כשכל העמוד נטען, כולל כל המשאבים.



2. מבני נתונים

מבני נתונים ב-Javascript



- כל מבני הנתונים הם סוגים שונים של אובייקטים

```
> let arr = [ 1, 2, "string1" ]  
⏏ undefined  
⏏  
> typeof(arr)  
⏏ 'object'  
⏏  
>
```

- ב-Javascript יש מבני נתונים שמגיעים עם השפה, כמו מערכים ואובייקטים.

- אפשר להטמיע מבני נתונים ע"י פונקציות (שהן גם אובייקטים), דוגמה להטמעת תור ע"י פונקציות בעמוד הבא

הטמעת תור ע"י פונקציות

```
8 function Queue() {
9   var collection = [];
10
11   this.print = () => {
12     console.log(collection);
13   }
14   this.enqueue = (element) => {
15     collection.push(element);
16   }
17   this.dequeue = () => {
18     return collection.shift();
19   }
20   this.front = () => {
21     return collection[0];
22   }
23   this.isEmpty = () => {
24     return collection.length === 0;
25   }
26   this.size = () => {
27     return collection.length;
28   }
29 }
```

```
31 let queue = new Queue();
32 queue.enqueue("first");
33 queue.enqueue("second");
34 queue.enqueue("third");
35
36 console.log(`size is: ${queue.size()}`);
37 console.log(`front is: ${queue.front()}`);
38 queue.print();
```

```
size is: 3
front is: first
▶ (3) ['first', 'second', 'third']
>
```

מערך

- מערך הוא אוסף של פריטים, מאותו סוג או מסוגים שונים, ששמורים יחד בזיכרון.
- האורך של המערך הוא דינמי - לא צריך להגדיר אותו בזמן האיתחול.
- get - אפשר לגשת לכל פריט ע"י האינדקס שלו, האינדקס מתחיל במיקום 0.
- ישנן מתודות מובנות בשפה, ששייכות לאובייקט מסוג מערך, כמו מיון, פילטור... הרשימה המלאה:
https://www.w3schools.com/js/js_array_methods.asp

- דוגמאות לשימוש במערכים:

```
8  const nums = [1, 2, "more"];
9  console.log(nums[0]);
10
11  let ids = [, , 223,];
12  console.log(ids[0]);
13  console.log(ids[2]);
14
15  ids[0] = 669;
16  console.log(ids[0]);
17  console.log(ids);
```


מערך

- מערך הוא אוסף של פריטים, מאותו סוג או מסוגים שונים, ששמורים יחד בזיכרון.
- האורך של המערך הוא דינמי - לא צריך להגדיר אותו בזמן האיתחול.
- get - אפשר לגשת לכל פריט ע"י האינדקס שלו, האינדקס מתחיל במיקום 0.
- ישנן מתודות מובנות בשפה, ששייכות לאובייקט מסוג מערך, כמו מיון, פילטור... הרשימה המלאה:
https://www.w3schools.com/js/js_array_methods.asp

- דוגמאות לשימוש במערכים:

```
8  const nums = [1, 2, "more"];
9  console.log(nums[0]);
10
11  let ids = [, , 223,];
12  console.log(ids[0]);
13  console.log(ids[2]);
14
15  ids[0] = 669;
16  console.log(ids[0]);
17  console.log(ids);
```

```
1
undefined
223
669
▶ (3) [669, empty, 223]
>
```

JSON

- מוגדר באובייקט מסוג json (ראשי תיבות של JavaScript Object Notation)
- ניתן להמרה למחרוזת בעזרת stringify או לאובייקט בעזרת parse
- משמש לשמירת נתונים מסוגים שונים, לשליחה על גבי http בין דפים/שרתים
- ל-json יש מבנה של key-value
- דוגמה לקובץ json:

Shenkar2024Keren > 6.1-JS-lec2 > json-example.json > ...

```
1  {
2    "menu": {
3      "id": "123",
4      "value": "File",
5      "popup": {
6        "menuitem": [
7          {"value": "New", "onclick": "CreateNewDoc()"},
8          {"value": "Open", "onclick": "OpenDoc()"},
9          {"value": "Close", "onclick": "CloseDoc()"}
10       ]
11     }
12   }
13 }
```

JSON - המשך

דוגמה להמרות מאובייקט למחרוזת ולהיפך:

```
8 let jsonStr = '{"id": "123", "value": "File", "action": "New file"}';
9 let jsonObj = JSON.parse(jsonStr); /* From string to object */
10
11 console.log(`jsonStr: ${jsonStr}`);
12 console.log(`jsonStr.action: ${jsonStr.action}`);
13
14 console.log(`jsonObj: ${jsonObj}`);
15 console.log(`jsonObj.id: ${jsonObj.id}`);
16 console.log(`jsonObj.action: ${jsonObj.action}`);
17
18 jsonObj.name = "added a name";
19 let jsonStrAgain = JSON.stringify(jsonObj); /* From object to string */
20
21 console.log(`jsonStrAgain: ${jsonStrAgain}`);
```

jsonStr: {"id": "123", "value": "File", "action": "New file"}	json.html:11
jsonStr.action: undefined	json.html:12
jsonObj: [object Object]	json.html:14
jsonObj.id: 123	json.html:15
jsonObj.action: New file	json.html:16
jsonStrAgain: {"id":"123","value":"File","action":"New file","name":"added a name"}	json.html:21

Custom objects

● אנחנו יכולים ליצור אובייקטים משלנו ולעבוד איתם, דוגמה:

```
8 let rectWidth = 20;
9 let rectangleObj = {width: rectWidth, height: 100};
10 console.log(rectangleObj.width); // GET
11
12 let songsObj = {
13   amount: 55,
14   private: true,
15   country: "Israel",
16   singers: ["Zohar", "Arik", "Aviv", "Rita"]
17 };
18
19 songsObj.singers[2] = "Shlomi"; // SET
20 songsObj.singers.push("Ofra");
21 console.log(songsObj);
```

מה יהיה הפלט של הקוד?

Custom objects (1)

● אנחנו יכולים ליצור אובייקטים משלנו ולעבוד איתם, דוגמה:

```
8 let rectWidth = 20;
9 let rectangleObj = {width: rectWidth, height: 100};
10 console.log(rectangleObj.width); // GET
11
12 let songsObj = {
13   amount: 55,
14   private: true,
15   country: "Israel",
16   singers: ["Zohar", "Arik", "Aviv", "Rita"]
17 };
18
19 songsObj.singers[2] = "Shlomi"; // SET
20 songsObj.singers.push("Ofra");
21 console.log(songsObj);
```

מה יהיה הפלט של הקוד?

20

```
▼ {amount: 55, private: true, country: 'Israel', singers: Array(5)} ⓘ
  amount: 55
  country: "Israel"
  private: true
  ▼ singers: Array(5)
    0: "Zohar"
    1: "Arik"
    2: "Shlomi"
    3: "Rita"
    4: "Ofra"
    length: 5
```

אובייקט קיים - style

● לכל אובייקט-תגית יש אובייקט style, המחזיק את מאפייני העיצוב שלו.

● לדוגמה: `document.getElementById("word").style.color = "blue";`

```
obj.style.backgroundColor = "black";  
obj.style.display = "none";  
obj.style.color = "#000000";  
obj.style.left = "10%";  
obj.style.top = "10px";  
obj.style.backgroundImage="url(logo.jpg)";  
obj.style.visibility = "hidden";  
obj.style.className = "title_over";
```

● עוד דוגמאות (שם האלמנט obj)

● ניתן להגיע לכל הגדרות העיצוב מהעמוד: <https://www.w3schools.com/cssref/index.php>



3. אירועים - events

אירוע - event



- נוצר כתוצאה מהתרחשות כלשהי, טריגר לדוגמה: מעבר עכבר מעל תמונה, לחיצה על קישור וכו'
- ההתרחשות הזו קוראת לאירוע שגורם לפעולה לדוגמה: מעביר את הגולש לכתובת אחרת, מחליף את הטקסט וכו'
- ישנם אירועים בשליטת המערכת, כמו טעינת העמוד onload
- ישנם אירועים בשליטת המשתמש, כמו לחיצה על אובייקט onclick
- תוכן יכול להשתנות לפי התנהגות המשתמש בממשק

חיבור אירוע לפונקציה - שיטה ישנה

אנחנו לא נתכנת ככה, זה ערבוב שכבות!

האירוע	מופעל כאשר	דוגמא
onfocus	מופעל כאשר אובייקט נבחר ומקבל מיקוד	<input type="text" onfocus ="dolt()">
onblur	מופעל כאשר אובייקט מאבד פוקוס לטובת אובייקט אחר	<input type="text" onblur ="dolt()">
onchange	אובייקט שמאבד פוקוס נבדק, כדי לדעת אם חל שינוי בערכו ההתחלתי, ואז מופעל האירוע	<input type="checkbox" onchange ="dolt()">
onclick	מתבצעת לחיצה על האובייקט	<input type="button" onclick ="dolt()">
onmouseover	מעבר עכבר על האובייקט	
onmouseout	יציאת עכבר מעל שטח האובייקט	
onload	יתרחש לאחר טעינת המסמך. מוגדר בתוך ה-body	<body onload ="dolt()">

חיבור אירוע לפונקציה - שיטה חדשה

```
index.html x  JS  scripts.js
Shenkar2024Keren > 6.1-JS-lec2 > 5C-Events > index.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <script src="js/scripts.js"> </script>
5      <title>2C - Javascript events</title>
6    </head>
7    <body>
8      <button id="btn">Click on me !</button>
9      <h1 id="pageTitle">Our products</h1>
10   </body>
11 </html>
```

- בקובץ חיצוני js נגדיר את ה-event

- אפשר ליעל עוד על ידי שימוש ב- addEventListener ובפונקציות self invoked

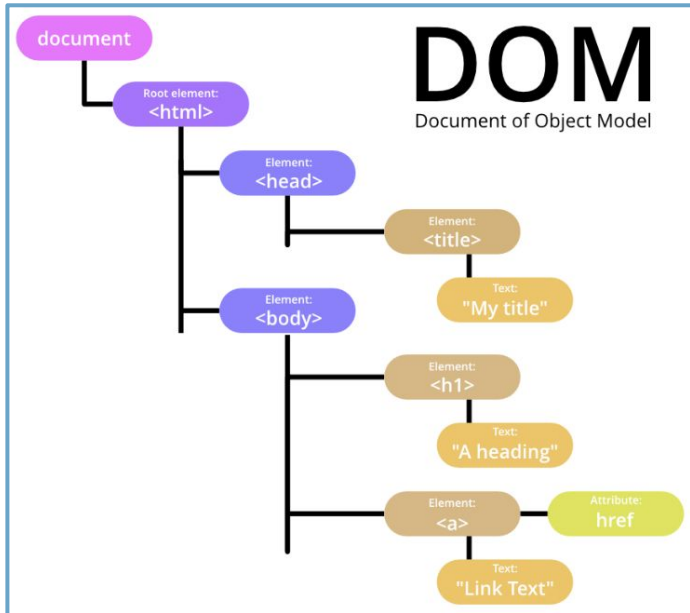
```
index.html  JS  scripts.js x
Shenkar2024Keren > 6.1-JS-lec2 > 5C-Events > js > scripts.js > ...
1  window.onload = () => {
2    |    document.getElementById("btn").onclick = changeTitle;
3  }
4
5  function changeTitle()
6  {
7    |    document.getElementById("pageTitle").innerHTML = "Our Services";
8  }
```



DOM-ה .4

DOM = Document Object Model

- הדפדפן סורק את התגיות שבמסמך HTML5 והופך כל תגית לצומת בתוך מבנה המכונה עץ DOM
- מבנה זה מאפשר להגדיר את הקשר ההיררכי והתלותי בין תגית אחת לתגית אחרת.
- כמו בעץ משפחתי, כך גם בעץ DOM ניתן להגדיר תגית או צומת ראשי שתחתיו ניתן להגדיר צמתים-בנים.
- צומת נקראת node. ה-node הראשי הוא השורש = root.
- מצומת השורש מסתעפים שאר הצמתים.



יחסים בין צמתים ב-DOM

- **אב ובן-**

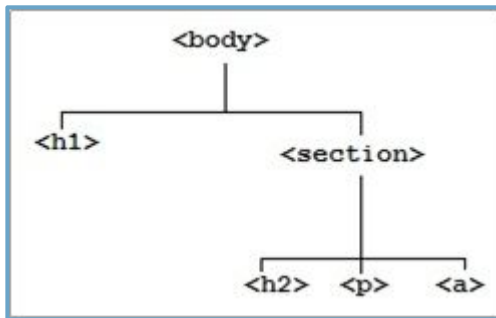
אם תגית מסוימת נמצאת בתוך תגית אחרת, הרי התגית הראשונה (הפנימית) היא תגית "בן" והתגית השנייה היא תגית "אב".

- **אחים-**

אם שתי תגיות נמצאות בתוך תגית שלישית, באותה היררכיה, הרי הם "אחים" זה לזה.

- **אב קדמון-**

כמו שבא מדור ראשון ומדור שני, כך "אב קדמון" הוא "אב של אב" בכל רמה שהיא.



נסתכל על הדוגמה, להמחשת המונחים.

שימושים לעומק ב-DOM



- ה-DOM מכיל elements, attributes ו-text
- טקסט שכתוב בתוך תגית הינו text node בתוך ה-node של תגית האב
-
- `let el = document.getElementById("news")` מהווה רפרנס לאובייקט ב-DOM
- אפשר לקרוא ל-`getElementsByTagName("a")` שמחזירה מערך של רפרנסים לכל התגיות מסוג זה.
- אפשר לקרוא ל-`getElementsByClassName("clear");` שמחזירה מערך של רפרנסים לכל התגיות עם ה-class הזה
- הרבה דוגמאות שקשורות ל-DOM תראו היום בתרגול



ES6 .5

ES6 new features

- 
1. Const keyword
 2. Let keyword
 3. Template literals
 4. Default parameter
 5. Spread operator
 6. Rest parameter
 7. Arrow functions
 8. Destructors
 9. Object property & methods shorthands
 10. Dynamic keys



Ecmascript = ES

- EcmaScript מגדירה את "ספר החוקים" (specification) של js, המפורסם ע"י Ecma International
- מוסיפה כלים שימושיים, משפרת את השפה ובין היתר עושה חיים קלים יותר למתכנתים.
- Ecmascript נוצרה כדי לייצר אחידות במימושים של js, משום שדפדפנים שונים יצרו סינטקס שונה עבור js.
- מאז ש-node.js יצאה לאוויר, הפופולריות של js גדלה משמעותית.
- <https://www.ecma-international.org/ecma-262/9.0/index.html>

Transpiler

- אי אפשר לסמוך על זה שכל הדפדפנים תומכים בפיצ'רים החדשים.
- לכן אנחנו צריכים לעשות Transpiling – לתרגם את הקוד משפת מכונה 1 לשפת מכונה 2
- נעזרים ב-Babel שמתרגם גירסאות חדשות יותר ל-ES5 (ניתן לכתוב ב-es7/8/9)
- דוגמאות להמרה מ-ES6 ל-ES5 (יהיה מובן יותר בהמשך):

BABEL

```
1 const myName = "Maluma";  
2 myName = "Juan Luis Londoño Arias";
```



```
1 "use strict";  
2  
3 function _readOnlyError(name) { throw new Error(`${name} is read-only`); }  
4  
5 var myName = "Maluma";  
6 myName = (_readOnlyError("myName"), "Juan Luis Londoño Arias");
```

ESNext / ES6+

- ESNext - שם גנרי שמתייחס לגירסה הבאה של JS.
- בגירסה ES6 (מהדורה 6, הנקראת גם ES2015) היתה קפיצה משמעותית
- הגירסה הנוכחית היא ES10 שיצאה בקיץ 2019, כאשר הגירסה הבאה בפיתוח.
- לעוד פרטים: [/https://www.javascripttutorial.net/es-next/](https://www.javascripttutorial.net/es-next/)

ES6 – 1. The 'const' keyword

- ע"י יצירת משתנה כ-const נגדיר משתנה לבלוק הנוכחי (scope of code) שהערך שלו **immutable** – בלתי ניתן לשינוי לאחר הגדרתו.
מכאן נובע שאנחנו חייבים לתת לו ערך בזמן ההגדרה.
- מגן מפני טעויות – אם ננסה לשנות את הערך, נקבל שגיאה מסוג `TypeError`.

```
const myName;  
// Uncaught SyntaxError: Missing initializer in const declaration
```

דוגמאות לא תקינות:

```
const myName = "Maluma";  
myName = "Juan Luis Londoño Arias";  
// Uncaught TypeError: Assignment to constant variable.
```

```
const scores = [75, 80, 95];  
  
for (const score of scores) {  
  | console.log(score);  
}  
// 75 80 95
```

דוגמה תקינה:

ES6 – 1. The 'const' keyword (1)

```
const colors = ['red'];  
colors.push('green');  
  
console.log(colors); // ["red", "green"]  
colors = []; // Uncaught TypeError: Assignment to constant variable
```

- הגדרת const ומערכים:

```
const person = { age: 20 };  
person.age = 30;  
  
console.log(person.age); // 30  
person = {age: 40}; // Uncaught TypeError: Assignment to constant variable
```

- הגדרת const ואובייקטים:

ES6 – 2. The 'let' keyword

ע"י יצירת משתנה כ-let אפשר להגדיר משתנה לבלוק הנוכחי (block scoped).
תזכורת – ע"י var יוצרים משתנה לפונקציה הנוכחית (function scoped)
בזמן hoisting המשתנה מוגדר אך לא מאותחל, כך שתיזרק שגיאה אם ננסה להשתמש בו.

במה הוא דומה ובמה הוא שונה מ-const ? מ-var ?

```
var x = 10;
if (x) {
  var x = 4; // overwrites the global var x
}
console.log(x); // 4

var y = 20;
if (y) {
  let y = 5; // only inside the if scope
}
console.log(y); // 20
```

ES6 – 3. Template literals

נקרא גם string interpolation.
נשים לב לסינטקס ולגרש הבודד (back tick) ולא ' (משתמשים במקש הטילדה)
כדי להכניס משתנה נשתמש בתבנית `${variable}`

מאפיינים ויתרונות:

- ❖ אפשר להשתמש בגרשיים או בגרש באמצע המחרוזת
- ❖ מאפשר ירידת שורה ללא שימוש בטכניקות שונות, כמו `join` או `\n`
- ❖ מאפשר לשרשר משתנים למחרוזות ללא הסימן `+`

```
const paragraph =  
  `This  
  text  
  can  
  span multiple lines`;  
  
console.log(paragraph);
```



```
This  
text  
can  
span multiple lines
```

```
const firstName = "Slim";  
const lastName = "Shady";  
console.log(`My name is ${firstName} ${lastName}`);
```



```
My name is Slim Shady
```

```
const num1 = 100;  
const num2 = 200;  
console.log(`${(num1 + num2)}`); // 300
```

דוגמאות:

ES6 – 4. Default parameters

קביעת ערכים דיפולטיביים לפרמטרים במידה ואינם קיימים או שהם undefined.

```
function sum(x = 0, y = 0) {  
  return x + y;  
}  
  
console.log(sum());           // 0  
console.log(sum(100));        // 100  
console.log(sum(undefined, 200)); // 200  
console.log(sum(100, 200));    // 300
```

אפשר להעביר פונקציות, לבצע חישובים...

```
const taxRate = () => 0.1;  
const getPrice = function( price, tax = price * taxRate() ) {  
  return price + tax;  
}  
  
const fullPrice = getPrice(100);  
console.log(fullPrice); // 110
```


ES6 – 4. Default parameters (1)

```
const doStuff = ( value = "default") => {  
  console.log(value)  
}
```

```
doStuff(undefined) // #1  
doStuff(false) // #2  
doStuff(NaN) // #3  
doStuff(null) // #4  
doStuff() // #5  
doStuff(0) // #6  
doStuff("") // #7
```

```
// ANSWERS  
// 1: "default"  
// 2: false  
// 3: NaN  
// 4: null  
// 5: "default"  
// 6: 0  
// 7: ""
```

נדגיש, 2 המצבים בהם יופעל הערך הדיפולטי:
אין ערך או undefined בלבד !

ES6 – 5. The Spread operator

ה-spread operator הוא 3 נקודות (לפני השם של הפרמטר) ומאפשר "לפתוח" מערך / אובייקט שיש לו איטרטור.

אפשר למקם היכן שנרצה, בדוגמה הראשונה הוא בתחילת המערך הגדול יותר. בין השאר, אפשר להשתמש בו כדי לאחד כמה מערכים, או להעתיק מערך

* נשים לב שהסינטקס דומה ל-Rest operator שנלמד בהמשך

```
const a = [1, 2, 3];  
const b = [...a, 4, 5, 6];  
console.log(b); // [1, 2, 3, 4, 5, 6]
```

```
const scores = [80, 70, 90];  
const copiedScores = [...scores];  
console.log(copiedScores); // [80, 70, 90]
```

```
const chars = ['A', ...'BC', 'D'];  
console.log(chars); // ["A", "B", "C", "D"]
```

ES6 – 6. The Rest parameter

ה-rest parameter הוא גם 3 נקודות (לפני השם של הפרמטר) ומאפשר לפונקציה לקבל מספר משתנים שאינו מוגדר מראש.

ימוקם תמיד בסוף רשימת הארגומנטים שהפונקציה מקבלת (אחרת נקבל שגיאה).

```
function myFunc(a, b, ...manyMoreArgs) {  
  console.log("a: " + a);  
  console.log("b: " + b);  
  console.log("manyMoreArgs: " + manyMoreArgs);  
}  
  
myFunc("one", "two", "three", "four", "five", "six");
```

a: one

b: two

manyMoreArgs: three,four,five,six

ES6 – 7. Arrow functions

יתרונות ומאפיינים:

- משמש פונקציות אנונימיות בלבד
- **חשוב!** בניגוד לפונקציה רגילה, לפונקציית חץ אין סקופ פנימי משלה
משמע: **inner this = outer this**
- במקרה שהפונקציה מקבלת פרמטר אחד – אפשר להוריד את הסוגריים עגולים.
בכל מקרה אחר – חייבים סוגריים עגולים
- במקרה שיש לפונקציה פקודה אחת – אפשר להוריד את הסוגריים המסולסלים.
בכל מקרה אחר – חייבים סוגריים מסולסלים
- החץ חייב להיות באותה השורה של ההצהרה, גוף הפונקציה יכול להיות בשורה מתחת.

```
const add = (x, y) => x + y;  
// const add = (x, y) => { return x + y; };  
console.log(add(10,20)); // 30;
```

ES6 – 7. Arrow functions (1)

```
function Car() {  
  this.speed = 0;  
  
  this.speedUp = function (speed) {  
    this.speed = speed;  
    setTimeout(function () {  
      console.log(this.speed); // undefined  
    }, 1000);  
  };  
}  
  
let car = new Car();  
car.speedUp(50);
```



```
function Car() {  
  this.speed = 0;  
  
  this.speedUp = function (speed) {  
    this.speed = speed;  
    let self = this;  
    setTimeout(function () {  
      console.log(self.speed);  
    }, 1000);  
  };  
}  
  
let car = new Car();  
car.speedUp(50); // 50;
```

```
function Car() {  
  this.speed = 0;  
  
  this.speedUp = function (speed) {  
    this.speed = speed;  
    setTimeout(  
      () => console.log(this.speed),  
      1000);  
  };  
}  
  
let car = new Car();  
car.speedUp(50); // 50;
```

ES6 – 8. Destructors

מאפשר לנו "לפרק" מערכים / אובייקטים בצורה מסודרת יותר

```
const conference = {  
  destination: "Rome conference",  
  travelers: 2,  
  field: "Web engineering 2019",  
  cost: 550  
}  
  
function getConfDetails( {destination, field} ) {  
  return `I will attend the ${destination} on ${field}`;  
}  
  
console.log(getConfDetails(conference));
```



ES6 – 8. Destructors (1)

```
function getProfile() {  
  return [  
    'John',  
    'Doe',  
    ['Red', 'Green', 'Blue']  
  ];  
}  
  
const [  
  firstName,  
  lastName,  
  [  
    color1,  
    color2,  
    color3  
  ]  
] = getProfile();  
  
console.log(color1, color2, color3); // Red Green Blue
```

ES6 – 9. Object **property** & methods **shorthands**

כשמאתחלים אובייקטים, ה-name יהיה כמו השם של המשתנה אם לא ציינו אחרת

```
function createMachine(name, status) {  
  return {  
    name: name,  
    status: status  
  };  
}
```



```
function createMachine(name, status) {  
  return {  
    name,  
    status  
  };  
}
```

בתוך אובייקט, אפשר להגדיר פונקציה ללא המילה function

```
const person = {  
  play: function() {  
    return "I like to play board games";  
  },  
  swim: function() {  
    return "I like to swim in the pool";  
  }  
};  
console.log(person.play()); // I like to play board games  
console.log(person.swim()); // I like to swim in the pool
```



```
const person = {  
  play() {  
    return "I like to play board games";  
  },  
  swim() {  
    return "I like to swim in the pool";  
  }  
};  
console.log(person.play()); // I like to play board games  
console.log(person.swim()); // I like to swim in the pool
```


ES6 – 10. Dynamic keys

הוספת שמות דינמית למפתחות באובייקט.

```
const key = 'name';  
const genKey = k => `index_${k*2}`;  
const obj = {};  
  
obj[key] = 'John Doe';  
obj[genKey(1)] = 21;  
console.log(obj); // { name: "John Doe", index_2: 21 }
```